Unit IV :: Data Visualisation and Plotting

SEC-01-PHY-91-R-03

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit IV: Data Visualisation and Plotting

2D plotting: using 'plot' command; customizing plots: markers, line styles; multiple plots: overlaying curves, combining datasets; 3D plotting basics: '3D plot' command, mesh and contour visualization applications: plotting oscillatory motion, damping curves, potential-well profiles.

1 Creating 2D plots

Creating clear 2D plots is an essential skill for visualising numerical results and communicating patterns quickly; in Scilab this typically means preparing paired data vectors `x` and `y` of equal length and calling `plot(x,y)` to draw the curve. By default plot produces a simple line plot, but multiple curves may be drawn together by supplying additional `(x,y)` pairs in the same call, which keeps axes consistent and avoids unintentionally overwriting earlier curves.

Good plots are annotated, i.e. we add an informative title and axis labels with `title`/`xtitle`, `xlabel` and `ylabel`, and identify multiple traces using `legend`. Enabling a `grid` (or `xgrid`) and choosing suitable line styles, markers and limits improves readability; finally, export figures to a suitable image or vector format for reports. These few steps – data preparation, plotting, annotation and exporting – form a practical workflow for most 2D visualisation tasks in Scilab.

1.1 Basic usage of the `plot()` function

The `plot()` function is the fundamental command for drawing 2-D line plots in Scilab. It accepts paired `(x,y)` vectors (of equal length) and draws y versus x; passing multiple `(x,y)` pairs in one call draws several curves on the same axes.

Plotting a single curve

A single curve is produced by supplying two vectors x and y of the same length. We use this when we want a simple plot of one dependent variable against an independent variable.

Example: *Plotting a simple sine curve*

```
x = 0:0.01:2*%pi;
y = sin(x);
plot(x, y);
```

Plotting multiple curves on the same plot

To draw several curves on the same axes, we pass additional `(x,y)` pairs to a single plot call (or the `plot()` function is called repeatedly while preserving the axes). Giving all pairs in one call is usually easiest and avoids creating extra figure windows.

Example: *Plotting sine and cosine curves on the same plot*

```
x = 0:0.01:2*%pi;
y1 = sin(x);
y2 = cos(x);
plot(x, y1, x, y2);
```

1.2 Adding labels, title and legend to a plot

Good plots communicate results clearly: axis labels, a descriptive title and a legend (when more than one curve appears) are essential. Scilab provides simple functions, such as `xlabel`, `ylabel`, `title` and `legend`, to add these annotations after drawing the plot.

The general syntax for each of these functions are given below:

- `xlabel('x-axis label');`
- `ylabel('y-axis label');`
- `title('Title of the plot');`
- `legend('label1', 'label2', ...);`

Example:

```
x = 0:0.01:2*%pi;
plot(x, sin(x), x, cos(x));
title('Trigonometric functions');
xlabel('x (rad)');
ylabel('f(x)');
legend('f(x)=sin(x)','f(x)=cos(x)');
```

2 Customizing plots

Customising plot appearance helps communicate data clearly. We can choose sensible line styles, markers and colours so that different series are easy to distinguish, and combine datasets thoughtfully when we want to compare or join results. Scilab gives flexible, scriptable control over these details, either by plotting several `(x,y)` pairs together or by modifying graphics object properties returned by `plot()`.

2.1 Markers and line styles

Plot markers (symbols at data points) and line styles (solid, dashed, dotted, etc.) make individual traces easy to identify and read. Use markers when showing discrete samples and use different line styles (and thicknesses) when curves might overlap; combine these with a legend so readers can tell traces apart.

Example:

```
x = 0:0.1:2*%pi;
y1 = sin(x);
y2 = sin(2*x);
ax1 = plot(x, y1, 'b');
ax2 = plot(x, y2, 'r');
title('Sine waves');
xlabel('x (rad)');
ylabel('f(x)');
legend('f(x)=sin(x)', 'f(x)=sin(2x)');
set(ax1, 'line_style', 1, 'thickness', 2, 'mark_style', 0);
set(ax2, 'line_style', 5, 'thickness', 1, 'mark_style', 4,
 'mark_size', 3);
```

2.2 Multiple plots

Multiple-plot techniques lets us compare datasets on the same axes or present several subplots. Two common tasks are overlaying different curves on the same axes and combining datasets (stitching or stacking) for continuous visualisation or joint analysis.

Overlaying curves

Overlaying means showing two or more curves on the same set of axes so they can be compared directly. The simplest and most robust way is to supply all `(x,y)` pairs to a single `plot()` call.

Example: *Overlaying two curves*

```
x1 = 0:0.01:1;
y1 = sin(2*%pi*x1);
```

```
x2 = 1.1:0.01:2;
y2 = 0.5*sin(4*%pi*(x2-1.1));
plot(x1, y1, x2, y2);
```

Combining datasets

Combining datasets can mean either concatenating two segments of a timeseries for a continuous plot or arranging data side-by-side for comparison. For a continuous (piecewise) plot, we concatenate the `x` and `y` vectors; for side-by-side comparison, we plot the separate series together and use distinct styles.

Example: *Stitching two datasets*

```
x1 = 0:0.01:1;
y1 = sin(2*%pi*x1);
x2 = 1.1:0.01:2;
y2 = 0.5*sin(4*%pi*(x2-1.1));
x_all = [x1, x2];
y_all = [y1, y2];
plot(x_all, y_all);
```

3 Basics of 3D plotting

3-D plotting lets us visualise surfaces, meshes and space curves so that structure in two-variable data or parametric trajectories is immediately visible.

Example: *A simple sinc surface*

This example builds a regular `(x,y)` grid, evaluates a smooth surface $Z = \frac{\sin r}{r}$ (with $Z = 1$ at $r = 0$), and displays it with axis labels and a grid.

```
x = -5.0:0.25:5.0;
y = -5.0:0.25:5.0;
[X, Y] = meshgrid(x, y);
R = sqrt(X.^2 + Y.^2);
Z = sin(R) ./ R;
Z(R==0) = 1;
plot3d(x, y, Z);
```

4 Applications

Applications of simple mechanical models provide immediate insight into oscillations, energy landscapes and dissipative behaviour encountered across physics and engineering. The plots and short scripts below illustrate three common topics: ideal (undamped) oscillatory motion which highlights sinusoidal time-dependence and frequency, damping curves that show how dissipation changes amplitude and introduces decay rates (underdamped, critically damped and overdamped cases), and potential-well profiles that visualise the forces and equilibrium points experienced by a particle.

4.1 Oscillatory motion

Given below is the analytic solution of the simple harmonic oscillator:

$$x(t) = A\cos\left(\omega t + \phi\right)$$

The Scilab code below plots a few cycles and also shows the phase-space trajectory $(x, \dot{x})$.

```
clear;
clf;

// parameters
A = 1.0;            // amplitude (m)
omega = 2*%pi*1;    // angular frequency (rad/s), here f = 1
   Hz
phi = 0;            // phase (rad)
T = 3;              // total time (s)
dt = 0.001;
t = 0:dt:T;

// solution and velocity
x = A * cos(omega.*t + phi);
v = -A * omega * sin(omega.*t + phi);

// time-series plot
subplot(2,1,1);
plot(t, x);
xtitle('Simple harmonic oscillator: x(t)', 'time (s)', 'x (m)
   ');
xgrid();

// phase-space plot (x vs v)
subplot(2,1,2);
plot(x, v);
xtitle('Phase-space trajectory','x (m)', 'v (m/s)');
xgrid();
```

4.2 Damping curves

Given below are the analytic solutions for a linear damped oscillator with mass m, damping coefficient c and stiffness k. Using $\gamma = \frac{c}{2m}$ and $\omega_0 = \frac{k}{m}$, we have

- Underdamping: $\gamma < \omega_0$
- Critical damping: $\gamma = \omega_0$
- Overdamping: $\gamma > \omega_0$

The Scilab code below plots each of the above three cases for comparison.

```
clear;
clf;
m = 1.0;          // kg
k = (2*%pi*1)^2 * m; // choose omega0 = 2*pi*1 rad/s (1 Hz
   natural frequency)
omega0 = sqrt(k/m);
A = 1.0;
phi = 0;
T = 6; dt = 0.001;
t = 0:dt:T;

// damping cases: underdamped, critical, overdamped
c_under = 0.5 * 2*m*omega0;   // gamma = 0.5*omega0 (
   underdamped)
c_crit  = 2*m*omega0;         // critical damping (gamma =
   omega0)
c_over  = 2.5 * 2*m*omega0;   // overdamped (gamma > omega0)

gamma_u = c_under/(2*m);
gamma_c = c_crit /(2*m);
gamma_o = c_over /(2*m);

// underdamped solution
omega_d = sqrt(max(0, omega0^2 - gamma_u^2)); // safe sqrt
x_u = A * exp(-gamma_u.*t) .* cos(omega_d.*t + phi);

// critically damped solution (one form)
x_c = A * (1 + omega0.*t) .* exp(-omega0.*t); // form with
   chosen initial conds for illustration

// overdamped (sum of exponentials) - choose coefficients to
   match initial conditions for illustration:
r1 = -gamma_o + sqrt(gamma_o^2 - omega0^2);
r2 = -gamma_o - sqrt(gamma_o^2 - omega0^2);
C1 = (A*r2)/(r2 - r1);
C2 = A - C1;
x_o = C1 * exp(r1.*t) + C2 * exp(r2.*t);
```

```
// plot
plot(t, x_u, t, x_c, t, x_o);
xtitle('Damped oscillator: comparison','time (s)','x(t) (m)')
   ;
legend('underdamped','critical','overdamped');
xgrid();
```

4.3 Potential well profiles

Some of the common 1D potential energy profiles used to visualise equilibrium points and barriers are:

a) Harmonic (quadratic) well $V(x) = \frac{1}{2}kx^2$

b) Finite square well of depth V_0 and width w

c) Symmetric double-well (quartic) potential $V(x) = ax^4 - bx^2$

The Scilab code below plots each of the above potential wells for comparison.

```
clear;
clf;
x = linspace(-3, 3, 1000);

k = 1.0;
V_harm = 0.5 * k .* x.^2; // harmonic potential

V0 = 5.0;
w = 1.0;
V_square = -V0 * (abs(x) <= w/2); // finite square well

a = 1.0; b = 3.0;
V_double = a .* x.^4 - b .* x.^2; //double-well potential

plot(x, V_harm, x, V_square, x, V_double);
xtitle('Potential well profiles','x (arb. units)','V(x) (arb.
    units)');
legend('harmonic (1/2 k x^2)', 'finite square well', 'double
   well (ax^4 - bx^2)');
xgrid();
```